ATELIER 2 Premier chrono

Algorithmie comparative - J2 - Lundi 9 novembre 2026, 9h44 - 50 minutes, en binôme

La question à laquelle l'atelier répond

Quand le nombre de colis est multiplié par dix, par combien le temps de l'affectation est-il multiplié, et quelles lignes du code de 2019 expliquent ce rapport ?

Ce que vous produisez : bench/resultats/j1-chrono.csv et sa courbe ; le constat entre dans l'ADR-001 de cet après-midi.

Le rendu attendu en fin d'atelier

Le correcteur de l'atelier répond sans KO :

```
OK  L'archive du référentiel est celle du module
OK  Le référentiel de la métropole est préparé à partir de cette archive
OK  Les jeux petite, moyenne et grande sont générés avec le référentiel du module
OK  Le legacy est chronométré aux trois tailles dans bench/resultats/j1-chrono.csv
OK  Les mesures sont cohérentes entre elles (même zone chronométrée)
OK  La courbe est tracée
6 contrôles, 0 KO
```

Pour démarrer

Le premier sous-objectif se fait ensemble, avec l'intervenant, pendant les dix premières minutes. Avant la mesure à 100 000 colis, lisez cette page et les sous-objectifs 1 à 3 ; pendant la mesure, la suite.

Sous-objectif 1, remettre le dépôt à jour : la première commande, à la racine du dépôt (le dossier efrei-logistique-perf).

Windows (PowerShell)

```
Expand-Archive -Path $HOME\Downloads\maj-J2.zip -DestinationPath . -Force
```

macOS et Linux

```
unzip -o ~/Downloads/maj-J2.zip -d .
```

Les sous-objectifs

1. Remettre le dépôt à jour (consigne 1 de la slide)

Pourquoi. La Base Adresse Nationale se régénère chaque nuit : toute la promotion part désormais de la même archive, celle du 29 octobre.

Première action. Décompressez maj-J2.zip, copiez l'archive dans data/referentiel, lancez `npm install` puis `npm run ban` : les quatre commandes de l'annexe, en un bloc.

Ce que vous devez voir. "archive du module : OK" puis "référentiel du module : OK".

2. Générer les trois jeux (consigne 2 de la slide)

Pourquoi. Trois tailles, chacune dix fois plus grande que la précédente : c'est ce qui permet de lire un rapport.

Première action. Lancez `npm run data -- --echelle petite`, puis moyenne, puis grande.

Ce que vous devez voir. Les nombres de colis, de doublons et de scans de l'annexe, à l'identique ; seule la durée change.

3. Chronométrer le legacy (consignes 3, 4 et 5 de la slide)

Pourquoi. Le premier rapport se lit entre 1 000 et 10 000 colis. La mesure à 100 000 colis prend une dizaine de minutes sur secteur (4 min 14 s sur le poste de capture), le double sur batterie : branchez-vous avant de la lancer.

Première action. Chronométrez 1 000 puis 10 000 colis, une fois Measure-Command (time sous macOS et Linux), calculez le premier rapport, lancez le jeu grande, puis préparez les deux questions ci-dessous pendant la mesure.

Ce que vous devez voir. affectes, non_affectes et doublons identiques à l'annexe ; Measure-Command (ou time) donne bien plus que duree_ms : retenez les deux, et celle que vous gardez pour la décision de l'atelier.

> **À préparer pendant la mesure à 100 000 colis, pour la mise en commun.**
> Dans le log du 24 septembre (pièce 04 de la data room, sur Moodle), combien vaut requetes_sql divisé par les 6 412 colis de la nuit (le log compte toutes les requêtes, doublons compris), et que compte ce nombre ?
> Quelle complexité annonce votre premier rapport ?

4. Lire la courbe, compléter l'hypothèse (consigne 6 de la slide)

Pourquoi. Un rapport proche de 10 annonce une croissance linéaire ; de 12 à 13, n log n ; de 100, quadratique. Le code ne s'ouvre qu'après.

Première action. Tracez la courbe (commande en annexe), puis divisez la médiane à 100 000 colis par celle à 10 000.

Ce que vous devez voir. Les trois médianes, et la courbe j1-chrono.svg ouverte dans le navigateur.

> **À préparer pour la mise en commun.**
> Le second rapport confirme-t-il votre hypothèse ?

5. Confirmer dans le code, vérifier (consignes 7 et 8 de la slide)

Pourquoi. Les trois réflexes du 1er octobre : les boucles imbriquées, les appels qui cachent une boucle, la structure choisie.

Première action. Ouvrez legacy/src/affectation.ts, relevez trois lignes (numéro et rôle), préparez la question de compréhension, puis lancez `npm run verifier -- 2`.

Ce que vous devez voir. Le correcteur finit sur "6 contrôles, 0 KO".

> **À préparer pour la mise en commun.**
> Quelles trois lignes de legacy/src/affectation.ts expliquent vos mesures : celle qui porte presque tout le coût, une autre qui grandit avec le nombre de colis, et la boucle qui explique votre division ?

> **La décision de l'atelier**
> Pendant la mesure : pour vos rapports, gardez-vous duree_ms ou la durée de Measure-Command (time sous macOS et Linux), et pourquoi ? Elle revient en rubrique 3 de l'ADR-001.

> **La question de compréhension**
> À quoi sert ce que vous venez de faire, en deux phrases ? Préparez-la à deux : à la mise en commun, deux binômes la disent, les autres jugent.

Les commandes de chaque sous-objectif, pour Windows, macOS et Linux, avec leur sortie attendue, sont dans l'annexe. À 17h00, le binôme commite : git add -A puis git commit -m "J2".

Si ça coince

- "L'exécution de scripts est désactivée" (Windows) : `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, puis fermez et rouvrez PowerShell.
- "archive absente" ou "n'est pas l'archive du module" : l'archive de Moodle n'est pas dans data/referentiel, ou c'est une autre version ; recopiez-la et relancez `npm run ban`.
- La mesure tourne encore à la minute 36 : laissez-la finir ; votre hypothèse est écrite : ouvrez le code et cherchez les lignes en attendant ; tracez la courbe et lancez le correcteur dès la fin de la mesure.

Toujours bloqué après ces trois pistes : levez la main, ou écrivez sur Teams. D'autres cas sont à la fin de l'annexe.

Si vous avez fini

- Refaites la division pour la nuit du 16 septembre (457 300 requêtes, 5 380 colis) : que concluez-vous sur le code de l'étape 2 ?
- Avec votre médiane à 10 000 colis et votre hypothèse, estimez le temps de calcul à 15 000 colis. Puis demandez-vous si c'est ce temps-là que la production paie : réponse cet après-midi.

Où trouver un exemple dans le code

legacy/src/batch.ts, lignes 36 et 59 : le chronomètre de l'étape, performance.now() avant et après l'appel ; c'est lui qui donne duree_ms. legacy/bin/chrono.ts ne chronomètre que le chargement des fichiers (chargement_ms). Ni l'un ni l'autre ne dit la complexité : c'est à vous de la lire.

Annexe : toutes les commandes, dans l'ordre

Toutes les commandes se lancent à la racine du dépôt. Les sorties montrées sont de vraies sorties, capturées sous Windows en jouant cette fiche ; vos durées seront différentes, vos nombres de colis, de couples et vos sommes de contrôle non. "Extrait" signale qu'on n'en montre que les lignes utiles.

1. Remettre le dépôt à jour

Windows (PowerShell)

```
Expand-Archive -Path $HOME\Downloads\maj-J2.zip -DestinationPath . -Force
Copy-Item $HOME\Downloads\adresses-33.csv.gz data\referentiel\
npm install
npm run ban
```

macOS et Linux

```
unzip -o ~/Downloads/maj-J2.zip -d .
cp ~/Downloads/adresses-33.csv.gz data/referentiel/
npm install
npm run ban
```

La sortie montrée est celle de npm run ban. npm install affiche le nombre de paquets ajoutés ; aucune ligne ne doit commencer par npm ERR!. Ceux qui ont fait la mise à jour chez eux lancent seulement npm run ban. Le dépôt du cours est aussi sur GitHub (adresse dans le README) : la même mise à jour s'obtient par git pull, le README donne les deux commandes.

Ce que vous devez voir (sortie réelle, Windows) :

```
archive du module : OK (adresses-33.csv.gz, version du 2026-10-29)
adresses-gironde.csv : 730137 adresses
adresses-metropole.csv : 229329 adresses (28 communes)
référentiel du module : OK (version du 2026-10-29)
```

2. Générer les trois jeux

Windows (PowerShell), macOS et Linux

```
npm run data -- --echelle petite
```

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
référentiel : 229329 adresses (adresses-metropole.csv)
jeu petite : 1000 colis (15 doublons, 66 pharmacie, 9 de plus de 25 kg), 739 clients, 9923
scans
```

Windows (PowerShell), macOS et Linux

```
npm run data -- --echelle moyenne
```

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
référentiel : 229329 adresses (adresses-metropole.csv)
jeu moyenne : 10000 colis (150 doublons, 812 pharmacie, 54 de plus de 25 kg), 7388 clients,
99662 scans
```

Windows (PowerShell), macOS et Linux

```
npm run data -- --echelle grande
```

Sur le poste de capture : 8,2 secondes.

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
référentiel : 229329 adresses (adresses-metropole.csv)
jeu grande : 100000 colis (1500 doublons, 7765 pharmacie, 510 de plus de 25 kg), 73875
clients, 996844 scans
écrit dans C:\cours\algo-captures-J2\efrei-logistique-perf\data\jeux\grande en 8.2 s
```

3. Chronométrer le legacy

Windows (PowerShell), macOS et Linux

```
npm run chrono -- --jeu petite --repetitions 3 --csv
```

Ce que vous devez voir (sortie réelle, Windows) :

```
[chrono] jeu=petite colis=1000 livreurs=85 chargement_ms=4
[chrono] etape=affectation repetition=1 duree_ms=21.3 colis=1000 tournees=85 affectes=985
non_affectes=0 doublons=15
[chrono] etape=affectation repetition=2 duree_ms=16.1 colis=1000 tournees=85 affectes=985
non_affectes=0 doublons=15
[chrono] etape=affectation repetition=3 duree_ms=16.1 colis=1000 tournees=85 affectes=985
non_affectes=0 doublons=15
[chrono] mediane_ms=16.1 min_ms=16.1 max_ms=21.3
[chrono] csv=bench\resultats\j1-chrono.csv lignes_ajoutees=3
```

Windows (PowerShell), macOS et Linux

```
npm run chrono -- --jeu moyenne --repetitions 3 --csv
```

Le premier rapport : la médiane à 10 000 colis divisée par la médiane à 1 000.

Ce que vous devez voir (sortie réelle, Windows) :

```
[chrono] jeu=moyenne colis=10000 livreurs=85 chargement_ms=15
[chrono] etape=affectation repetition=1 duree_ms=1424.3 colis=10000 tournees=85 affectes=9850
non_affectes=0 doublons=150
[chrono] etape=affectation repetition=2 duree_ms=1402.1 colis=10000 tournees=85 affectes=9850
non_affectes=0 doublons=150
[chrono] etape=affectation repetition=3 duree_ms=1409.0 colis=10000 tournees=85 affectes=9850
non_affectes=0 doublons=150
[chrono] mediane_ms=1409.0 min_ms=1402.1 max_ms=1424.3
[chrono] csv=bench\resultats\j1-chrono.csv lignes_ajoutees=3
```

Windows (PowerShell)

```
Measure-Command { npm run chrono -- --jeu petite | Out-Default }
```

macOS et Linux

```
time npm run chrono -- --jeu petite
```

Measure-Command sous Windows, time sous macOS et Linux. La sortie montrée est celle de Windows ; time affiche real, user et sys sous la sortie du chrono : real est la durée à noter. Cette mesure n'ajoute rien au CSV.

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
[chrono] etape=affectation repetition=1 duree_ms=20.3 colis=1000 tournees=85 affectes=985
non_affectes=0 doublons=15
TotalMilliseconds : 519,6006
```

Windows (PowerShell), macOS et Linux

```
npm run chrono -- --jeu grande --repetitions 1 --csv
```

Une dizaine de minutes sur secteur (4 min 14 s sur le poste de capture), le double sur batterie : branchez-vous avant de la lancer. Rien d'autre ne tourne sur ce poste pendant la mesure : une deuxième mesure partagerait le processeur. N'ouvrez pas le code : lisez la fiche, faites la division, écrivez l'hypothèse.

Ce que vous devez voir (sortie réelle, Windows) :

```
[chrono] jeu=grande colis=100000 livreurs=85 chargement_ms=150
[chrono] etape=affectation repetition=1 duree_ms=253725.9 colis=100000 tournees=85
affectes=14784 non_affectes=83716 doublons=1500
[chrono] csv=bench\resultats\j1-chrono.csv lignes_ajoutees=1
```

4. Lire la courbe, compléter l'hypothèse

Windows (PowerShell), macOS et Linux

```
npm run courbe -- bench/resultats/j1-chrono.csv --log --ouvrir
```

--ouvrir ouvre la courbe dans le navigateur ; sans lui, ouvrez bench/resultats/j1-chrono.svg à la main.

Ce que vous devez voir (sortie réelle, Windows) :

```
fichier : bench\resultats\j1-chrono.csv
  legacy     taille=   1000  mediane_ms=      16.1  (3 mesures)
  legacy     taille=  10000  mediane_ms=    1409.0  (3 mesures)
  legacy     taille= 100000  mediane_ms=  253725.9  (1 mesure)
courbe écrite : bench\resultats\j1-chrono.svg
```

5. Confirmer dans le code, vérifier

Windows (PowerShell)

```
code legacy\src\affectation.ts
```

macOS et Linux

```
code legacy/src/affectation.ts
```

Sans VS Code, ouvrez le fichier dans votre éditeur. Relevez le numéro de chaque ligne et ce qu'elle fait grandir : c'est ce qu'un binôme lira à la mise en commun.

Windows (PowerShell), macOS et Linux

```
npm run verifier -- 2
```

Le correcteur relit le CSV, la courbe et le référentiel ; les questions se disent à la mise en commun.

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
OK  L'archive du référentiel est celle du module
OK  Le référentiel de la métropole est préparé à partir de cette archive
OK  Les jeux petite, moyenne et grande sont générés avec le référentiel du module
OK  Le legacy est chronométré aux trois tailles dans bench/resultats/j1-chrono.csv
OK  Les mesures sont cohérentes entre elles (même zone chronométrée)
OK  La courbe est tracée
6 contrôles, 0 KO
```

Cas particuliers

- Le correcteur dit qu'un jeu a été "généré avec un autre référentiel" : il date d'avant la mise à jour ; régénérez-le.
- Vos téléchargements ne sont pas dans $HOME\Downloads (Windows) ou ~/Downloads (macOS) : adaptez le chemin. Sur un Linux en français, le dossier s'appelle souvent ~/Téléchargements.
- Le navigateur a décompressé l'archive (adresses-33.csv au lieu de adresses-33.csv.gz) : retéléchargez-la sans l'ouvrir ; `npm run ban` veut le .gz.
- Linux : unzip absent, installez-le (`sudo apt install unzip` sur Ubuntu).
- macOS : la commande code est introuvable. Dans VS Code, palette de commandes, "Shell Command: Install 'code' command in PATH" ; ou ouvrez le fichier dans votre éditeur.